





BUILDING SECURE CONTAINER IMAGES

DEVOPSDAYS ZURICH

ALVARO REVUELTA



ABOUT ME

- Alvaro Revuelta.
- Systems Developer
- Laboratory for Life Sciences - SciLifeLab
- Get the code (and slides)





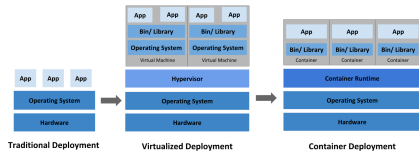
PRESENTATION STRUCTURE

The explanation on the slides is interluded with the explanations in the README in the repository

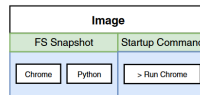
1. Introduction
2. Image Structure
3. How layers are generated
4. Optimize images
5. Conclusion



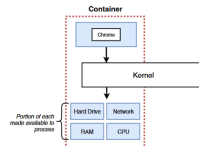
1. INTRODUCTION



(Source: [Kubernetes.io](https://kubernetes.io))



(a) image



(b) container

(Source: [Kubernetes.io](https://kubernetes.io))



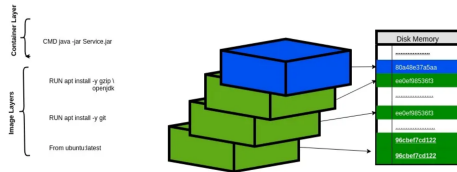
1. INTRODUCTION

- Images are **immutable**. Once an image is created, it can't be modified.
- Container images are composed of **layers**. Each layer represents a set of file system changes that add, remove, or modify files.



2. IMAGE STRUCTURE

- Images consist of **multiple layers** stacked on top of each other.
- Layers are **file system changes** (e.g., file additions, deletions, updates).
- Layers are **read-only**; only the top container layer is **writable** at runtime.
- Layers are merged to form a **Unified File System** (Full explanation would need another presentation).



(Source: [UnionFS : A File System of a Container](#))



3. HOW IMAGES ARE GENERATED

- Layers are **reusable** in subsequent builds, because they are cached.
- **FROM:** Setting the base image (starting point)
- **RUN:** Run a command (e.g., installing software)
- **COPY/ADD:** Adding files
- **CMD/ENTRYPOINT:** Setting the default command for running the container
- RUN, ADD and COPY generate new layers.

Image hierarchy			
FROM	debian:12, 12.7, bookworm, bookworm-20240926, latest		1
FROM	buildpack-deps:bookworm-curl, curl, stable-curl		1
FROM	buildpack-deps:bookworm-scm, scm, stable-scm		1
FROM	buildpack-deps:bookworm		2
ALL	node:latest		1
Layers (13)			
0	ADD file:087f68d558e06c7160c9322582925635e7539a7702413828357c28c77f6f345 in /	49.56 MB	1
1	CMD ["bash"]	0 B	✓
2	/bin/sh -c set -eux; apt-get update; apt-get install -y --no-install-recommends ca-certificates curl ...	24.05 MB	1
3	/bin/sh -c set -eux; apt-get update; apt-get install -y --no-install-recommends git mercurial opens...	64.39 MB	1
4	/bin/sh -c set -eux; apt-get update; apt-get install -y --no-install-recommends autoconf automake b...	211.27 MB	1
5	RUN /bin/sh -c groupadd --gid 1000 node && useradd --uid 1000 --gid node --shell /bin/bash --cre...	3.32 KB	✓
6	ENV NODE_VERSION=22.9.0	0 B	✓
7	RUN /bin/sh -c ARCH= && dpkgArch="\$(dpkg --print-architecture)" && case "\$dpkgArch#" in i...	53.86 MB	✓
8	ENV YARN_VERSION=1.22.22	0 B	✓
9	RUN /bin/sh -c set -ex && export GNUPGHOME="\$(mktemp -d)" && for key in 6A010C516600659...	1.25 MB	✓
10	COPY docker-entrypoint.sh /usr/local/bin/ # buildkit	444 B	✓

(Source: [DockerHub](#))



4. OPTIMIZE IMAGES

trivy image imagetag:version

```
/ # trivy image rv0lt/flaskrediswebapp:basic | head
2023-04-21T18:04:09.297Z      INFO    Vulnerability scanning is en
2023-04-21T18:04:09.297Z      INFO    Secret scanning is enabled
2023-04-21T18:04:09.297Z      INFO    If your scanning is slow, pl
2023-04-21T18:04:09.297Z      INFO    Please see also https://aqua
ret detection
2023-04-21T18:04:11.456Z      INFO    Detected OS: debian
2023-04-21T18:04:11.456Z      INFO    Detecting Debian vulnerabili
2023-04-21T18:04:11.691Z      INFO    Number of language-specific
2023-04-21T18:04:11.691Z      INFO    Detecting python-pkg vulnera

rv0lt/flaskrediswebapp:basic (debian 10.13)
=====
Total: 146 (UNKNOWN: 5, LOW: 86, MEDIUM: 21, HIGH: 32, CRITICAL: 2)
```



(github.com/aquasecurity/trivy)



4. OPTIMIZE IMAGES

Minimizing the Number of Layers:

- Combine multiple RUN commands into one (e.g., chaining commands using &&).
- Removing unnecessary files during the build process (e.g., apt-get clean).
- Avoid using COPY to add files that won't be needed during runtime.

Using Smaller Base Images:

- Alpine Linux vs. Ubuntu: Trade-offs between size and functionality.
- Multi-stage builds: Using one image for building the application and another for the final image (stripped of development tools).

Caching Layers Efficiently:

- Take advantage of layer caching by ordering Dockerfile instructions logically.
- Place less frequently changed commands earlier in the Dockerfile to maximize caching.

Removing Unnecessary Packages and Files:

- Remove temporary files, build dependencies, or logs to reduce image size.
- Clean up the file system after installations (e.g., `rm -rf /var/lib/apt/lists/*`).



4. OPTIMIZE IMAGES

Subsequent building times: +60s

```
Dockerfile basic X Dockerfile basic2 X Do
basic > Dockerfile > ...
1 # use ubuntu as base image
2 FROM ubuntu
3
4 # copy the source code
5 COPY hello.c hello.c
6
7 # install build-essential package to compi
8 RUN apt update
9 RUN apt install -y build-essential
10
11 # Compile and generate binary
12 RUN gcc -o helloWorld hello.c
13
14 # Run the program
15 ENTRYPOINT ["/helloWorld"]
16
17
```

Subsequent building times: 1s

```
Dockerfile basic Dockerfile basic2 X Do
basic2 > Dockerfile > ...
1 # use ubuntu as base image
2 FROM ubuntu
3
4 # install build-essential package to compi
5 RUN apt update
6 RUN apt install -y build-essential
7
8 # copy the source code
9 COPY hello.c hello.c
10
11 # Compile and generate binary
12 RUN gcc -o helloWorld hello.c
13
14 # Run the program
15 ENTRYPOINT ["/helloWorld"]
16
17
```



4. OPTIMIZE IMAGES

```
# use ubuntu as base image
FROM ubuntu as build-env

# install build-essential package to compile the source
RUN apt update && apt install -y build-essential

# copy the source code
COPY hello.c hello.c

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# FROM alpine for an even greater size reduce
FROM ubuntu

# copy binary executable to new layer
COPY --from=build-env ./helloWorld ./helloWorld

# Run the program
ENTRYPOINT ["/helloWorld"]
```

WEIGHT IN MB REDUCED BY 4X TIMES



4. Optimize images - Alpine

- Alpine images are lightweight and can significantly reduce container sizes.
- However, they come with limitations, particularly regarding determinism.
- Alpine removes older versions of packages from its repositories, potentially breaking reproducibility. → [GitLab discussion](#)



4. OPTIMIZE IMAGES - DISTROLESS

- If every change in the file system creates a new layer. Can there be an empty image?
- Yes! Introducing **FROM SCRATCH**
- Images built using this base are called **distroless**.
- By reducing the image size, you are also reducing the **attack surface** for possible vulnerabilities.



4. OPTIMIZE IMAGES - DISTROLESS

- Even though there is a considerable increase in security and performance, images built this way introduce a considerable **development overhead**.
- For learning purposes they are pretty useful, but for production ready environment maybe not the best.
- Google realized this some years back and developed their own **suite of distroless images** ([link to Google Distroless repository](#)).

```
# use ubuntu as base image
FROM ubuntu as build-env

# install build-essential package to compile the source code
RUN apt update && apt install -y build-essential

# copy the source code
COPY hello.c hello.c

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# use distroless container to run the program
FROM SCRATCH

# copy the required libraries to run the program
COPY --from=build-env /lib/aarch64-linux-gnu/libc.so.6 /lib/aarch64-linux-gnu/libc.so.6
COPY --from=build-env /lib/ld-linux-aarch64.so.1 /lib/ld-linux-aarch64.so.1

# copy the /etc/passwd
COPY --from=build-env /etc/passwd /etc/passwd

# tmp directory
COPY --from=build-env /tmp /tmp

# copy binary executable to new layer
COPY --from=build-env ./helloWorld ./helloWorld
```

(Source: own)



4. OPTIMIZE IMAGES - DISTROLESS

- In the quest for optimizing images, some projects have appeared.
- To debug them in platform such as Kubernetes, one can use [ephemeral containers](#).
- Ubuntu Chiselled images ([link](#)).
- COPA project ([link](#)).



5. OTHER TOOLS FOR SECURITY

- Kube-hunter: Cluster Misconfiguration Detection; Pen-testing.
 - Same makers as Trivy
 - github.com/aquasecurity/kube-hunter
- Kubescape: similar than kube-hunter.
 - github.com/kubescape
- Kube-linter: Same as Trivy, but to find misconfiguration in k8s yaml files.
- Falco: Runtime Security.

Vulnerabilities
For further information about a vulnerability, search its ID in:
<https://avd.aquasec.com/>

ID	LOCATION	MITRE CATEGORY	VULNERABILITY	DESCRIPTION	EVIDENCE
None	Local to Pod (kube-hunter-075v6)	Lateral Movement // ARP poisoning and IP spoofing	CAP_NET_RAW Enabled	CAP_NET_RAW is enabled by default for pods. If an attacker manages to compromise a pod, they could potentially take advantage of this capability to perform network attacks on other pods running on the same node	
KHWR02	10.96.0.1:443	Initial Access // Exposed sensitive interfaces	K8s Version Disclosure	The kubernetes version could be obtained from the /version endpoint	v1.27.2



VERY INTERESTING LECTURE



National Security Agency
Cybersecurity and Infrastructure Security Agency

Cybersecurity Technical Report

Kubernetes Hardening Guide

KUBERNETS HARDENING GUIDE



CNCF LANDSCAPE



OFFICIAL COMPLIANT KUBERNETES [CNCF.IO](https://cncf.io)



6. CONCLUSION

Key Takeaways

- Importance of understanding image layers and their role in optimization.
- Best practices for reducing image size and improving efficiency.
- Tools and techniques for analyzing and improving images.

Next steps?

1. Learn the basics of containers and building images → This Workshop
2. Learn about container registries and CICD pipelines
 - Introduction to GitOps, [Trivy action](#)
 - Create images for every PR or feature branch to test it out
 - We can talk about docker alternatives.
3. Deploy in a Kubernetes cluster
 - Advanced GitOps, [ArgoCD](#)
 - [SealedSecrets](#)



6. CONCLUSION

CONTACT:

LINKEDIN - [ALVARO REVUELTA MARTINEZ](#)
ALVARO.REVUELTA@SCILIFELAB.UU.SE



6. CONCLUSION

Q & A